

This is a work in progress. I think I'm on the right track, but don't have a solution yet. It's basically a brute force evaluation of the K4 text according to Friedman's analytical approach for WHEATSTONE ciphers. I'm able to derive millions of putative solutions (alphabets) that generate the K4 ciphertext and cribs; I was unable to get putative solutions when I tried to analyze a randomly chosen set of 97 letters as an alternate ciphertext. That leads me to believe that WHEATSTONE was in fact used for K4.

My problem is I have a wealth of putative solutions, and need some way to prune. Any suggestions welcomed.

If K4 uses a WHEATSTONE device, we should be able to fit the data (CT letters and cribs) to the device. Back in 1918, Friedman showed how to go about doing this (see Riverbank Pub 20). He showed a paper and pen approach, assuming he had 25 or so messages to work with; we have (I believe) 3 messages of 33 letters. We now have computers for handling trial and error, but having only 97 letters is a real challenge (leading to many putative solutions with no easy way to rule many of them out).

Step 1: Determine basic parameters

The most common WHEATSTONE structure is a 27 element plaintext alphabet and a 26 element cipher alphabet, generally inscribed on discs or rotor-like wheels. In K4 we have 97 cipher text letters that use 26 letters, so it's likely that our cipher wheel has 26 elements. In WHEATSTONE we have a plain alphabet containing the 26 A-Z letters plus one more, which is typically a word space character. But we have cribs that contain no word space characters, so the choice of the 27th letter is not certain.

The wheels rotate together in lock step. The cycle length is 702, meaning that after 702 steps the wheels are back in the original starting position. We can label the number of steps from a starting point as the "wheel index" which runs from 0 to 701 before returning to 0. Each wheel index corresponds to a specific plain/cipher letter pair. From a given index, the wheels step up to 26 times, so only the next 26 wheel positions are reachable from any given index.

Step 2: Determine the number of cycles

A cycle consists of a fixed P/C starting position (specific starting wheel index) and some number of letter encipherments until a reset back to the starting position. If we have sentence delimiters in the text, each sentence is a cycle; at the end of the sentence the wheels are reset and the next sentence is then encrypted.

With a cycle length of 702, and an average of 13 or so steps between adjacent message letters, the sentence length is typically chosen to be somewhat less than 50 to avoid cycling (from 701 back to 0). (See Friedman).

K4 doesn't have explicit sentence delimiters, so it probably uses fixed length blocks of letters as a "sentence". Looking at the K4 cribs, it's likely that K4 is blocked into sentences of 33 letters. A step value of 26 corresponds to a full rotation of the cipher wheel, leading to ciphertext doubles. The

ciphertext SS at 32 implies the wheels step 26 times between 32 and 33 and that the corresponding crib letters ST are adjacent in the plain alphabet; the plaintext pair ST at 23 should likewise have a step of 26, but with ciphertext letters RV that is not the case. A likely resolution to this apparent conflict is that the wheels were reset after the first 33 letters were encrypted.

The argument for a reset after 66 letters is based on preliminary analysis. The beginning of each sentence should correspond to low numbered wheel index values, and the end of each sentence should correspond to high index values. This condition is met if we assume VL at 66 begins a cycle, and VT at 24 is toward the end of cycle 1.

Step 3: Fill out the Cipher and Plain alphabets

Choose any P/C pair such as VL at 66 and put the V in position 0 of the cipher alphabet and the L in position 0 of the plain alphabet. All subsequent additions to the alphabets are relative to this arbitrary starting value. Label the wheel index for this VL wheel pair as 26 (an arbitrary choice, since subsequent wheel positions are all relative to this arbitrary starting point). Since VL is the first letter enciphered at the start of cycle 3, the Initial Position is somewhere within 26 positions from VL (somewhere between 0 and 25).

Now, following Friedman's approach, chain the given P/C data together to determine some of the alphabet contents. Given the VL at 66, how many times could the wheels step to produce ZL at 70? Between 66 and 67 is 1 to 25 steps, between 67 and 68 is exactly 26 steps because we have ciphertext doublet TT, between 68 and 69 is 1 to 25 steps, and between 69 and 70 is 1 to 25 steps. Thus between 66 and 70 we have between 29 and 101 steps. Further, the L on the plain wheel repeats after 27 steps, so between 66 and 70 we could only have 54 or 81 steps. Try these 2 guesses successively. For each guess, put the plain letter in the 27 element plain alphabet, the ciphertext letter in the 26 element cipher alphabet, and label the message letter (0..96 for K4) with the corresponding windex (wheel index 0..701).

Next, note that the VL at 66 and the VT at 24 must be a multiple of 26 steps; try all 27 possibilities. For each guess, ...

Given VT at 24, how many steps could lead to RT at 28. Again, between 29 and 101 steps, subject to the constraint that the T at 24 and the T at 28 must be a multiple of 27 steps, so again 54 or 81.

Step 4: Chain when you can, guess when you can't

The goal is to fill out the 26 element cipher alphabet. Chaining lets us fill in the alphabet positions. For vacant positions in the alphabet, guess one of the unassigned cipher letters from A-Z. Look for the same cipher letter in the message and iteratively repeat the chaining process.

As you proceed, validate the partial results within each cycle. Ensure that the steps between adjacent letters is in the range 1..25 (or 26 if the ciphertext is a doublet). As conflicts occur, back up to the next guess and repeat.

Notes

Remember that the plaintext is pre-encoded to remove plaintext doubles (the system uses 1 to 26 steps, and 27 would be needed to encode a plaintext doublet so these are removed ahead of time). Plaintext doubles are replaced, typically EE would be replaced by EQ or QE or EJ or JE etc. If our first crib is preceded by an E, the plaintext could get converted to something like QASTNORTHEAST; if it's followed by T it could get preencoded as EASTNORTHEASQ.

There's a possible exception to this: since we don't know the 27th letter on the plain alphabet, it's possible that any of the 26 A-Z letters could be repeated. The 27th letter could be a punctuation letter that doesn't show up in our cribs (period, question, etc.) or it could be one of the A-Z letters duplicated (the 27 element plain alphabet could contain two E's or two T's or whatever).

I was not able to get a solution for the crib EASTNORTHEAST; I get many solutions for EASTNORTHEASQ. Either the last letter is part of a plaintext doublet TT (encoded as QT, as ILLUSION gets encoded as IQLUSION), or the 27th letter of the plain alphabet is a second T.

Filter the Results

Here's where I'm stuck: at the end of this process I end up with millions of putative cipher alphabets. I can fill out the plain alphabet for the crib letters, but the rest of the positions are unknown. So for each putative solution we have a partial decrypted plaintext. The crib letters account for about 70% of letters in common English, so we have somewhere around 70% of the plaintext for each putative solution.

I then made the assumption that the cipher alphabet is derived from a keyword; there are multiple ways of converting a keyword to an alphabet.

- a. write the keyword, followed by the remaining letters
- b. write the keyword, eliminating duplicate letters, followed by the remaining letters in rows beneath the keyword. Read the alphabet down the columns going left to right.
- c. same as b, but read the columns in alphabetical order (for keyword KRYPTOS read the columns under KOPRSTY)
- d. same as b, but don't eliminate the duplicate letters when filling out the rows. Then cross off the duplicate letters in the top row (the keyword), and read down the columns

I tried taking a word dictionary and encoding each word in the above 4 methods, and then comparing the putative results against these known keyword alphabets, but so far no luck. The putative alphabets were all shifted to account for the fact that we have a relative alphabet (first letter being assigned arbitrarily).

So, either the keyword was not encoded via one of the four methods above, or the keyword is not in my dictionary, and it is something like a name or location. I'm working on an alphabet-to-keyword

converter: guess the keyword length, reconstruct the keyword matrix, and check the 2nd row to see if it's a bunch of letters in sequence.

An Example

The below cipher/plain alphabet pair (one of several million viable alphabet solutions) can reproduce the K4 ciphertext and the associated crib EASTNORTHEASQ and BERLINCLOCK. Column 1 is the letter (0..96), col2 is the ciphertext letter and col 3 is the crib letter, col 4 is the wheel index 0..701, and col 5 and 6 are the offsets into the cipher and plain alphabets for this index, col 7 is the decrypted plaintext, and col 8 is the number of wheel steps from the prior letter.

In this example, I arbitrarily chose the P/C pair at 33 as the anchor (cipher alphabet[0] = S and plain alphabet[0] = Q. I arbitrarily set the wheel index of this P/C pair to 26, since we know the real resync point is somewhere within 26 of the first letter of each cycle.

View in a text editor or spreadsheet with courier font, landscape mode, to see the 3 cycles for the 33 letter sentences.

```
cipher wheel[26]          SACMHPUBLEJNOKTFWDYVGRZIXQ
plaintext wheel [27]      QC---E-T-KS-ONI---R-L--HAB-
```

Cycle 1								Cycle 2				
								Cycle 3				
Letter	CT	Crib	Index w26	w27	PT	Steps	Letter	CT	Crib	Index w26	w27	
	Index w26	w27	PT	Steps	Letter	CT	Crib	Index w26	w27	PT	Steps	
0	O	-	12	12	13	N		33	S	Q	26	0
	0	Q		66	V	L	19	19	20	L		
1	B	-	33	7	7	T	21	34	O	-	38	12
	12	O	12	67	T	I	40	14	14	I	21	
2	K	-	39	13	13	N	6	35	T	-	40	14
	14	I	2	68	T	N	66	14	13	N	26	
3	R	-	47	21	21	-	8	36	W	-	42	16
	16	-	2	69	M	C	81	3	1	C	15	
4	U	-	58	6	5	E	11	37	T	-	66	14
	13	N	24	70	Z	L	100	22	20	L	19	

5	O	-	64	12	11	-	6	38	Q	-	77	25
	24	A	11	71	F	O	119	15	12	O	19	
6	X	-	76	24	23	H	12	39	S	-	78	0
	25	B	1	72	P	C	135	5	1	C	16	
7	O	-	90	12	10	S	14	40	J	-	88	10
	8	-	10	73	K	K	143	13	9	K	8	
8	G	-	98	20	18	R	8	41	Q	-	103	25
	23	H	15	74	W	-	146	16	12	O	3	
9	H	-	108	4	1	C	10	42	S	-	104	0
	24	A	1	75	G	-	150	20	16	-	4	
10	U	-	110	6	3	-	2	43	S	-	130	0
	23	H	26	76	D	-	173	17	12	O	23	
11	L	-	112	8	5	E	2	44	E	-	139	9
	5	E	9	77	K	-	195	13	7	T	22	
12	B	-	137	7	3	-	25	45	K	-	143	13
	9	K	4	78	Z	-	204	22	16	-	9	
13	S	-	156	0	22	-	19	46	Z	-	152	22
	18	R	9	79	X	-	206	24	18	R	2	
14	O	-	168	12	7	T	12	47	Z	-	178	22
	17	-	26	80	T	-	222	14	7	T	16	
15	L	-	190	8	2	-	22	48	W	-	198	16
	10	S	20	81	J	-	244	10	2	-	22	
16	I	-	205	23	17	-	15	49	A	-	209	1
	21	-	11	82	C	-	262	2	20	L	18	
17	F	-	223	15	8	-	18	50	T	-	222	14
	7	T	13	83	D	-	277	17	8	-	15	
18	B	-	241	7	26	-	18	51	J	-	244	10
	2	-	22	84	I	-	283	23	14	I	6	
19	B	-	267	7	25	B	26	52	K	-	247	13
	5	E	3	85	G	-	306	20	10	S	23	
20	W	-	276	16	7	T	9	53	L	-	268	8
	26	-	21	86	K	-	325	13	2	-	19	
21	F	E	301	15	5	E	25	54	U	-	292	6
	23	H	24	87	U	-	344	6	21	-	19	

22	L 7	A T	320 11	8 88	24 H	A -	19 368	55 4	D 18	- R	303 24	17
23	R 13	S N	333 6	21 89	10 U	S -	13 370	56 6	I 20	- L	309 2	23
24	V 17	T -	357 4	19 90	7 A	T -	24 391	57 1	A 14	- I	313 21	1
25	Q 5	N E	363 15	25 91	13 U	N -	6 396	58 6	W 19	- -	328 5	16
26	Q 12	O O	389 7	25 92	12 E	O -	26 399	59 9	I 22	- -	335 3	23
27	P 26	R -	395 14	5 93	18 K	R -	6 403	60 13	N 26	- -	349 4	11
28	R 3	T -	411 4	21 94	7 C	T -	16 418	61 2	F 14	- I	353 15	15
29	N 21	H -	427 18	11 95	23 A	H -	16 443	62 1	B 12	- O	371 25	7
30	G 25	E B	436 4	20 96	5 R	E -	9 463	63 21	N 5	B E	375 20	11
31	K 5	A E	455 7	13	24	A	19	64	Y	E	382	18
32	S 18	S R	468 13	0	10	S	13	65	P	R	395	5